



**Pontificia Universidad
Católica del Ecuador**
Seréis mis testigos

MANABÍ

[Integrante 1]

[Integrante 2]

[Integrante 3]

[Integrante 4]

[Integrante 5]

RePoints — Diseño de la Solución de IA

Asignatura:

Sistemas Expertos — Desarrollo y Evaluación de SE

CARRERA DE DESARROLLO DE SOFTWARE

Semestre: Quinto

Profesor:

[Nombre del profesor]

Julio, 2026

1. Definición del problema y objetivos

1.1 Problema

Actualmente la separación de residuos de plástico y vidrio dentro del campus universitario se realiza de forma manual, dependiendo del criterio de cada persona. Esto provoca contaminación cruzada entre materiales reciclables y reduce la eficiencia de los procesos de reciclaje posteriores. RePoints propone automatizar esta clasificación mediante visión por computador embebida en una Raspberry Pi 5, apoyada por un módulo de sistema experto que valide la predicción del modelo antes de aceptarla como definitiva.

1.2 Objetivo general

Diseñar una solución de inteligencia artificial capaz de clasificar en tiempo real residuos de plástico y vidrio a partir de una imagen capturada por cámara, integrando un modelo de visión por computador con un módulo de reglas (sistema experto) que valide la confianza de cada predicción antes de tomar una decisión final.

1.3 Objetivos específicos

- Seleccionar y justificar una arquitectura de red neuronal adecuada para clasificación binaria de imágenes bajo restricciones de hardware embebido (Raspberry Pi 5, sin GPU dedicada).
- Definir un flujo de entrenamiento reproducible (partición de datos, hiperparámetros, función de pérdida y optimizador) a partir del dataset limpio y dividido documentado en el informe de análisis del dataset.
- Establecer una estrategia de validación basada en métricas estándar de clasificación que permita detectar sobreajuste y sesgo entre clases.
- Identificar los riesgos técnicos propios del entorno real de despliegue (iluminación variable, fondos no controlados, materiales transparentes) y definir estrategias de mitigación.

1.4 Alcance

Tipo de clasificación: Binaria (plástico vs. vidrio), con manejo adicional de una categoría operativa "desconocido" cuando la confianza del modelo es insuficiente.

Entradas: Imagen RGB de un residuo, capturada por una cámara USB conectada a la Raspberry Pi 5, en el momento en que el objeto se coloca frente al sensor.

Salidas: Etiqueta de clase (plástico / vidrio / desconocido), probabilidad de confianza asociada, y registro del resultado en un dashboard web para su trazabilidad.

Fuera de alcance: Clasificación de otros materiales (metal, papel, orgánico) y control físico de actuadores de separación, que corresponden a otras materias del proyecto integrado y no a este diseño.

2. Arquitectura del sistema

La figura 1 presenta el flujo completo de la solución, desde la captura física de la imagen hasta la decisión final validada y su registro.

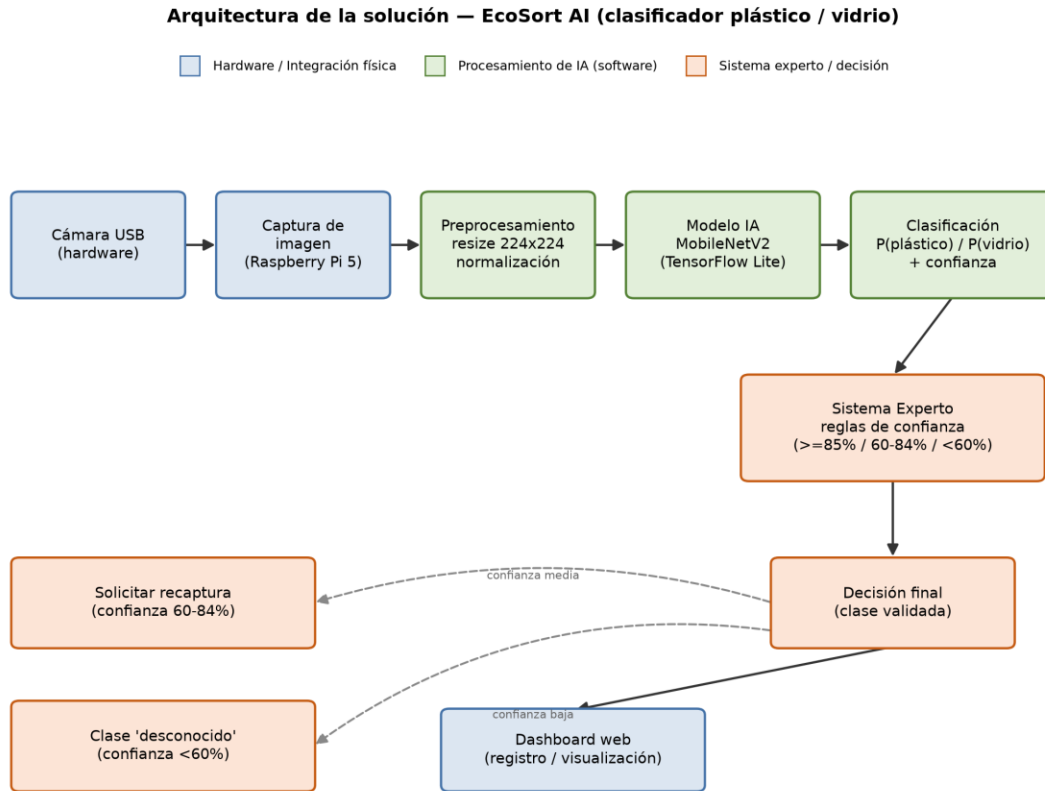


Figura 1. Arquitectura de la solución RePoints: pipeline de captura, preprocesamiento, inferencia, validación por sistema experto y salida.

2.1 Descripción de componentes

- Cámara USB (hardware): sensor de captura conectado físicamente a la Raspberry Pi 5; adquiere la imagen del residuo cuando se activa la rutina de captura.
- Captura de imagen (Raspberry Pi 5): proceso en la propia placa que lee el frame de la cámara y lo entrega al pipeline de software para su procesamiento.
- Preprocesamiento: redimensiona la imagen a 224×224 px (entrada esperada por MobileNetV2) y normaliza los valores de píxel al rango que exige el preprocesamiento de la red preentrenada.
- Modelo de IA (MobileNetV2, exportado a TensorFlow Lite): recibe el tensor preprocesado y produce una probabilidad por clase (plástico / vidrio).
- Clasificación: combina la clase de mayor probabilidad con su nivel de confianza asociado, insumo directo para el sistema experto.
- Sistema experto (reglas de confianza): valida la predicción aplicando umbrales — confianza $\geq 85\%$ acepta la clase directamente; 60-84% solicita una recaptura de la imagen; $< 60\%$ marca el resultado como "desconocido" en vez de forzar una clasificación poco confiable.
- Decisión final e integración con hardware/dashboard: una vez validada, la clase se registra y visualiza en un dashboard web, quedando disponible como evidencia para las materias de Tecnologías de Plataforma y Manejo y Desarrollo de Proyectos del mismo proyecto integrador.

Esta separación entre "modelo que predice" y "sistema experto que decide" es intencional: permite ajustar el umbral de aceptación sin reentrenar el modelo, y da trazabilidad explícita a los casos límite (confianza media/baja) en vez de forzar siempre una respuesta binaria.

3. Modelo seleccionado y justificación

3.1 Alternativas consideradas

Modelo	Ventaja principal	Motivo de descarte / uso
CNN propia (desde cero)	Control total de la arquitectura	Requeriría mucho más datos y tiempo de entrenamiento para igualar la precisión de un modelo preentrenado; el dataset actual (2670 imágenes) es pequeño para entrenar desde cero de forma robusta.
MobileNetV2 (transfer learning)	Muy liviana, diseñada para dispositivos embebidos; excelente relación precisión/costo computacional	Seleccionada — ver justificación en 3.2.
MobileNetV3	Ligeras mejoras de eficiencia sobre V2	Alternativa válida de segunda opción; se prioriza V2 por mayor soporte y documentación estable en TensorFlow Lite.
EfficientNet (B0)	Mejor precisión relativa a su tamaño en benchmarks generales	Mayor costo computacional y de memoria que MobileNetV2 para una ganancia marginal en un problema binario relativamente simple.
YOLO	Excelente para detección con localización (bounding boxes)	Diseñado para detección de múltiples objetos en una escena; este proyecto es clasificación de un único objeto centrado, por lo que añadiría complejidad innecesaria.
Edge Impulse (plataforma)	Entrenamiento y despliegue guiado sin código	Útil como alternativa de prototipado rápido, pero se prefiere control directo del pipeline (Python/TensorFlow) para ajustar hiperparámetros con precisión.

3.2 Justificación de MobileNetV2

- Recursos computacionales: la Raspberry Pi 5 no cuenta con GPU dedicada; MobileNetV2 usa convoluciones separables en profundidad (depthwise separable convolutions), lo que reduce drásticamente el número de parámetros (~3.4M) y operaciones frente a redes clásicas (VGG, ResNet), permitiendo inferencia en CPU en tiempos de cientos de milisegundos.
- Precisión esperada: para un problema binario con clases visualmente distinguibles (forma, color, transparencia), el transfer learning desde ImageNet ya provee filtros de bajo/medio nivel (bordes, texturas, transparencias) reutilizables, por lo que se espera una precisión de validación alta (>90%) incluso con un dataset de tamaño moderado.
- Tiempo de inferencia: al exportar el modelo entrenado a TensorFlow Lite (formato .tflite) se habilita cuantización (float16 / int8), reduciendo aún más el tamaño del modelo y el tiempo de inferencia en la Raspberry Pi, un requisito crítico para una clasificación en tiempo real.
- Factibilidad de implementación: MobileNetV2 tiene soporte de primera clase en TensorFlow/Keras y en TensorFlow Lite, con abundante documentación y ejemplos de despliegue en Raspberry Pi, lo que reduce el riesgo técnico de esta fase del proyecto.

4. Flujo de entrenamiento

4.1 División del dataset

Se reutiliza la partición estratificada 70/15/15 (entrenamiento/validación/prueba) documentada y ya generada en el informe de análisis del dataset, con semilla fija (random_state=42) para garantizar reproducibilidad:

Conjunto	Porcentaje	Plástico	Vidrio	Total
Entrenamiento	70%	1006	862	1868
Validación	15%	215	184	399
Prueba	15%	217	186	403
Total	100%	1438	1232	2670

4.2 Hiperparámetros e implementación

Parámetro	Valor definido	Justificación breve
Épocas	30 (con early stopping, paciencia=5 sobre val_loss)	Suficientes para converger con transfer learning; el early stopping evita sobreajuste.
Batch size	32	Balance estándar entre estabilidad del gradiente y uso de memoria en entrenamiento (Google Colab / GPU compartida).
Función de pérdida	Binary Crossentropy	Apropiada para clasificación binaria con salida sigmoide (plástico=1 / vidrio=0).
Optimizador	Adam	Convergencia rápida y estable sin necesidad de ajuste manual extenso de la tasa de aprendizaje.
Learning rate	1e-4 inicial, con ReduceLROnPlateau (factor 0.5, paciencia=3)	Tasa conservadora adecuada para fine-tuning de un modelo preentrenado; se reduce automáticamente si la validación se estanca.
Estrategia de transfer learning	Fase 1: base MobileNetV2 congelada, se entrena solo el clasificador superior. Fase 2: se descongelan las últimas ~30 capas para fine-tuning con learning rate reducido (1e-5)	Evita destruir los filtros preentrenados al inicio y permite especialización progresiva hacia el dominio del dataset.
class_weight	Balanceado (clase vidrio con mayor peso)	Compensa el desbalance leve (1438 plástico vs. 1232 vidrio) detectado en el análisis del dataset.
Data augmentation en entrenamiento	Rotación $\pm 20^\circ$, flip horizontal, variación de brillo/contraste $\pm 20\%$, zoom ligero	Simula variabilidad de fondo/iluminación ausente en el dataset original (ver sección 6).

5. Estrategia de validación del modelo

Dado que el dataset presenta un desbalance leve entre clases (1438 vs. 1232), la exactitud (accuracy) por sí sola puede ocultar un desempeño pobre en la clase minoritaria. Por ello se define un conjunto de métricas complementarias, todas calculadas sobre el conjunto de prueba (15%, aislado durante todo el entrenamiento):

Métrica	Qué mide	Por qué es apropiada aquí
Accuracy	Proporción total de predicciones correctas	Métrica general de referencia, fácil de comunicar en el informe final.
Precision (por clase)	De lo que el modelo predijo como una clase, cuánto era correcto	Relevante porque una clase mal predicha (p. ej. vidrio marcado como plástico) puede contaminar el flujo de reciclaje.
Recall (por clase)	De todos los casos reales de una clase, cuántos detectó el modelo	Importante para la clase minoritaria (vidrio), donde interesa no perder casos reales por el desbalance.
F1-score	Media armónica de precision y recall	Resume el balance entre ambas en una sola cifra, útil para comparar variantes del modelo (con/sin fine-tuning, con/sin augmentation).
Matriz de confusión	Distribución completa de aciertos/errores entre clases	Permite ver directamente si los errores se concentran en una dirección (p. ej. vidrio transparente confundido con plástico PET transparente), el riesgo identificado en el análisis del dataset.

Adicionalmente, se monitorearán las curvas de accuracy y loss (entrenamiento vs. validación) por época durante todo el entrenamiento, para detectar sobreajuste tempranamente (divergencia entre ambas curvas) y decidir si es necesario reforzar el data augmentation o reducir la complejidad del fine-tuning.

6. Riesgos técnicos

Riesgo	Impacto	Estrategia de mitigación
Iluminación variable en el punto de uso real	El dataset de origen fue capturado con luz interior homogénea; luz natural/sombras pueden degradar la precisión	Data augmentation de brillo/contraste; capturas propias en la universidad bajo distintas condiciones de luz.
Fondo y ángulo de captura muy repetitivos en el dataset (mantel / piso de madera, toma cenital)	Riesgo de que el modelo memorice el fondo en vez del objeto (shortcut learning) y falle ante fondos nuevos	Diversificar fondos en la captura propia; aplicar recorte/zoom aleatorio como augmentation adicional.
Objetos parcialmente visibles o rotos	Menor información visual disponible para la clasificación	Incluir ejemplos de este tipo en el conjunto de entrenamiento; el sistema experto puede marcar baja confianza como "desconocido" en vez de forzar una clase.
Similitud entre materiales transparentes (PET transparente vs. vidrio claro)	Es el caso límite más probable de confusión entre las dos clases	Priorizar estos ejemplos en la captura adicional; revisar específicamente estos casos en la matriz de confusión.

Riesgo	Impacto	Estrategia de mitigación
Desbalance leve de clases (1438 vs. 1232)	Sesgo leve del modelo hacia la clase mayoritaria (plástico)	class_weight balanceado durante el entrenamiento; augmentation algo más intensivo sobre la clase vidrio.
Sobreajuste (overfitting) por dominio limitado del dataset de origen	Buen desempeño en el test set derivado de Kaggle pero pobre en producción	Early stopping sobre val_loss; validar también sobre un pequeño conjunto de fotos reales tomadas en la universidad antes de dar por aceptado el modelo.

7. Estrategias de mejora

- Data augmentation: rotación, flip horizontal, variación de brillo/contraste y zoom aleatorio aplicados en tiempo de carga, para compensar la baja variabilidad de fondo/iluminación del dataset original.
- Transfer learning en dos fases: entrenamiento inicial con la base MobileNetV2 congelada seguido de fine-tuning de las últimas capas, para adaptar los filtros preentrenados al dominio específico de residuos de plástico y vidrio sin perder el conocimiento general de ImageNet.
- Ajuste de hiperparámetros: búsqueda acotada sobre learning rate, número de capas descongeladas y peso de clases, comparando resultados mediante F1-score en el conjunto de validación antes de fijar la configuración final.
- Captura de imágenes reales dentro de la universidad: complementar el dataset de Kaggle con fotografías tomadas con la misma cámara USB y Raspberry Pi 5 del proyecto, en el entorno real de despliegue, usadas como conjunto adicional de validación "del mundo real" antes de aceptar el modelo como definitivo.

Referencias

Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., & Chen, L.-C. (2018). MobileNetV2: Inverted Residuals and Linear Bottlenecks. CVPR 2018. <https://arxiv.org/abs/1801.04381>

TensorFlow. (s.f.). TensorFlow Lite — Deploy ML models on mobile and edge devices. <https://www.tensorflow.org/lite>

Informe interno: Análisis del Dataset de Proyectos Integradores — RePoints (PUCE Manabí, 2026), documento previo de este mismo proyecto integrado.